

Informe De Estado Tecnico Y Hoja De Ruta

Plataforma de datos PostgreSQL, backend Spring Boot y siguientes incrementos del sistema de gestion para tienda de abarrotes.

Campo	Valor
Proyecto	Tienda de Abarrotes
Codigo	TDA-EST-TEC-001
Version	v0.1
Estado	En revision
Responsable	Por definir
Fecha de corte	2026-07-23
Clasificacion	Interno

12 Migraciones Flyway	2 Endpoints de negocio	24/24 Pruebas exitosas	UP Backend y PostgreSQL
--------------------------	---------------------------	---------------------------	----------------------------

Resultado del corte

La plataforma de datos se encuentra definida y operativa en Docker. El backend cuenta con una primera vertical completa del modulo de catalogo: crear una marca y consultarla por UUID. Los clientes Angular y movil, la seguridad y el resto de modulos de negocio permanecen pendientes.

Control De Versiones

Version	Fecha	Autor	Cambio	Estado
v0.1	2026-07-23	Equipo del proyecto	Creacion inicial del informe de estado y hoja de ruta.	En revision

1. Resumen Ejecutivo

El proyecto se estructuro como monorepo para tres aplicaciones: un backend Spring Boot, una futura aplicacion web Angular y una futura aplicacion movil. La prioridad inicial fue construir una plataforma de datos transaccional y auditable antes de desarrollar los clientes.

PostgreSQL 18.4 se ejecuta mediante Docker Compose con pgAudit 18.0. El esquema se evoluciona exclusivamente con Flyway y Hibernate se limita a validar que el modelo JPA coincida con la base. El backend utiliza Java 21, Spring Boot 4.1.0, Gradle Kotlin DSL y arquitectura hexagonal modular.

1.1 Estado Por Componente

Componente	Estado	Evidencia	Siguiente accion
Repositorio y estructura	Implementado	apps, database, docs, scripts, deploy y data.	Mantener limites y actualizar documentacion.
Plataforma PostgreSQL	Operativa	12 migraciones; contenedor healthy; puerto local 5433.	Evolucionar solo mediante nuevas migraciones.
Backend Spring Boot	En desarrollo	Crear y consultar marcas; Actuator; 24 pruebas exitosas.	Listar, actualizar y desactivar marcas.
Seguridad	Pendiente	Esquema iam existe; no hay Spring Security ni autenticacion.	Definir login, tokens, roles y permisos.
Web Angular	Pendiente	Carpeta reservada; aplicacion no generada.	Iniciar cuando el contrato API base sea estable.
Aplicacion movil	Pendiente	Carpeta reservada; aplicacion no generada.	Definir plataforma y estrategia offline.
Produccion y CI/CD	Pendiente	Docker local de datos; backend no esta en Compose.	Dockerfile, pipeline, secretos y observabilidad.

Lectura correcta del avance

La base de datos esta preparada para mas capacidades que las implementadas actualmente en Java. La existencia de un esquema o tabla no significa que el correspondiente caso de uso del backend ya este disponible.

1.2 Objetivo Funcional Vigente

- Gestionar usuarios, roles y permisos sin almacenar contraseñas en texto plano.
- Administrar catalogo, inventario, compras, ventas, caja, credito y facturacion.
- Soportar varias sucursales, almacenes, lotes, caducidad y operacion offline controlada.
- Proteger inventario y documentos comerciales mediante transacciones, restricciones e idempotencia.
- Mantener auditoria funcional append-only y auditoria tecnica con PostgreSQL y pgAudit.

2. Estructura Y Arquitectura

2.1 Organizacion Del Monorepo

Ruta	Responsabilidad	Estado
apps/backend	API Spring Boot y reglas de negocio.	Activo
apps/web	Futura aplicacion Angular.	Reservado
apps/mobile	Futura aplicacion movil.	Reservado
database/migrations	Fuente canonica del esquema Flyway.	Activo
database/seeds	Datos ficticios para desarrollo.	Activo
database/tests	Pruebas SQL de integridad y transacciones.	Activo
docs	Requisitos, arquitectura, ADR, operacion y releases.	Activo
scripts	Automatizacion PowerShell de tareas locales.	Activo
deploy/postgres	Imagen PostgreSQL con pgAudit.	Activo
data/local-only	Datos locales no versionables.	Local

2.2 Arquitectura Hexagonal Modular

El backend es un monolito modular. Cada modulo de negocio puede tener dominio, aplicacion, puertos y adaptadores propios. La direccion permitida de dependencias es adapter -> application -> domain.

Cientes Angular / movil	->	REST Controladores y DTO	->	Aplicacion Casos de uso y puertos
			->	Dominio Entidades e invariantes
		JPA Adaptador de persistencia	<-	Puerto de salida Repositorio
		PostgreSQL Esquemas y restricciones	<-	

2.3 Tecnologias Aprobadas E Incorporadas

Tecnologia	Version / uso	Proposito
Java	21	Lenguaje del backend.
Spring Boot	4.1.0	API, inyeccion, configuracion y Actuator.
Gradle	Kotlin DSL + wrapper	Construccion reproducible sin Maven.
Spring Data JPA	Gestionada por Spring Boot	Adaptador ORM.
PostgreSQL	18.4	Persistencia transaccional.
pgAudit	18.0	Auditoria tecnica de ROLE y DDL.
Flyway	12.11.0	Migraciones versionadas.
MapStruct	1.6.3	Maapeo REST y persistencia.
Lombok	Gestionado por Spring Boot	Reduccion de codigo repetitivo.
Testcontainers	Gestionado por Spring Boot	Pruebas contra PostgreSQL real.
Docker Compose	Local	Base, migraciones, herramientas y pruebas SQL.

3. Plataforma De Datos Implementada

3.1 Principios Y Esquemas

La base utiliza una sola instancia PostgreSQL con separacion por esquemas funcionales. Los agregados sincronizables usan UUID; las fechas se almacenan con zona horaria y se normalizan a UTC; importes y cantidades usan tipos NUMERIC.

Esquema	Responsabilidad
organization	Sucursales, almacenes y dispositivos.
iam	Usuarios, roles, permisos y cuentas PostgreSQL.
catalog	Productos, presentaciones, marcas, categorias, precios e impuestos.
inventory	Saldos, lotes, movimientos, reservas, cupos y conteos.
purchasing	Proveedores, compras y cuentas por pagar.
sales	Clientes, carritos, pedidos, pagos, devoluciones y credito.
cash	Cajas, sesiones y movimientos de efectivo.
billing	Datos fiscales y comprobantes CFDI preparados.
sync	Inbox, outbox, secuencias y conflictos de sincronizacion.
audit	Auditoria funcional, tecnica y revisiones de acceso.

3.2 Migraciones Versionadas

Version	Archivo	Capacidad principal
V001	foundation	Extensiones, funciones, tipos y base comun.
V002	audit_and_database_principals	Auditoria y cuentas de PostgreSQL.
V003	organization	Sucursales, almacenes y dispositivos.
V004	identity_and_authorization	Usuarios, roles y permisos.
V005	catalog	Marcas, productos, presentaciones y precios.
V006	inventory	Existencias, lotes, reservas y movimientos.
V007	purchasing	Proveedores, compras y recepciones.
V008	sales_and_credit	Ventas, pagos, devoluciones y credito.
V009	cash	Cajas, sesiones y movimientos.
V010	billing_mexico	Preparacion de datos para CFDI 4.0.
V011	synchronization	Idempotencia, inbox, outbox y conflictos.
V012	views_indexes_and_privileges	Vistas, indices y privilegios finales.

Regla de evolucion
database/migrations es la fuente canonica. Las migraciones aplicadas no se editan; todo cambio estructural requiere una nueva version. Hibernate usa ddl-auto=validate y no crea tablas.

4. Concurrencia, Transacciones Y Auditoria

4.1 Proteccion Contra Sobreventa

La solucion acordada para dos compradores que intentan adquirir la ultima unidad no depende de una lectura previa aislada. La reserva o descuento debe ejecutarse como actualizacion condicional dentro de una transaccion.

```
UPDATE inventory.stock_balances
SET on_hand_quantity = on_hand_quantity - :quantity,
    version = version + 1
WHERE warehouse_id = :warehouse_id
    AND product_presentation_id = :presentation_id
    AND available_quantity >= :quantity;
```

Una fila afectada significa que la operacion obtuvo inventario. Cero filas significa existencia insuficiente. Si una venta contiene varios renglones y cualquiera falla, la transaccion completa debe ejecutar rollback. El commit se realiza solo cuando todos los renglones y efectos asociados son validos.

4.2 Reglas De Negocio Protegidas

Codigo	Regla
RN-001	Disponible = fisico - reservado - asignado a dispositivos.
RN-002	El inventario disponible nunca puede ser negativo.
RN-003	Una operacion multirrenglon confirma o revierte todos sus efectos.
RN-004	El costo se calcula mediante promedio ponderado.
RN-005	La salida fisica sugerida de lotes sigue FEFO.
RN-006	Una clave de idempotencia repetida no duplica efectos.
RN-007	Clientes offline solo preparan carritos; no confirman pagos.
RN-008	Empleados offline venden solo contra cupos asignados.
RN-009	Ventas, compras, movimientos y auditorias no se eliminan.
RN-010	Las bitacoras no guardan secretos ni datos completos de pago.

4.3 Auditoria En Dos Capas

- Funcional: tablas append-only en el esquema audit para eventos de negocio.
- Tecnica: pgAudit para operaciones ROLE y DDL, logs nativos y conciliacion de cuentas.
- Principio de seguridad: usuarios de negocio y cuentas de PostgreSQL son conceptos distintos.
- Retencion: debe definirse una politica de volumen, rotacion y consulta de bitacoras.

5. Backend Implementado

5.1 Configuración Base

Elemento	Implementación actual
Perfiles	local, dev, prod y test.
Secretos	Variables de entorno y archivo .env ignorado por Git.
Conexión local	PostgreSQL en localhost:5433; base tienda_abarrotes.
JPA	open-in-view=false; ddl-auto=validate; UTC.
Servidor	Puerto 8080 por defecto; apagado graceful.
Actuador	health e info; detalle completo solo en local.
Pool	Hikari con límites diferenciados por perfil.
Mapeo	MapStruct para REST y persistencia.
Persistencia de pruebas	Testcontainers con PostgreSQL y migraciones reales.

5.2 Primera Vertical Del Módulo Catalogo

La entidad Brand es Java puro y no depende de Spring o JPA. Normaliza el código a mayúsculas, valida longitud y caracteres, exige nombre, genera UUID y fecha de creación, e inicia en estado ACTIVE.

Capa	Tipos principales	Responsabilidad
Dominio	Brand, BrandStatus, InvalidBrandException	Invariantes y normalización.
Aplicación	CreateBrandService, GetBrandByIdService	Orquestar casos de uso.
Puertos	CreateBrandUseCase, GetBrandByIdUseCase, BrandRepositoryPort	Contratos de entrada y salida.
REST	BrandController, DTO y BrandRestMapper	Contrato HTTP y validación de entrada.
Persistencia	BrandJpaEntity, BrandPersistenceAdapter	Mapeo y acceso a catalog.brands.
Errores	BrandExceptionHandler	Mapeo a 400, 404 y 409.
Compartido	ApiResponseDto<T> record	Envoltura uniforme de respuestas de negocio.

5.3 Contrato Común De Respuesta

ApiResponseDto es un record inmutable. Separa el estado HTTP, un código estable para clientes, el mensaje humano, los datos, errores de campos y la ruta. Las fábricas success y error validan que no se mezclen datos con errores.

```
{
  "timestamp": "2026-07-23T00:00:00Z",
  "status": 200,
  "code": "BRAND_FOUND",
  "message": "Marca encontrada correctamente",
  "data": { ... },
  "errors": null,
  "path": "/api/v1/catalog/brands/{uuid}"
}
```

6. API Disponible

6.1 Base URL Local

```
http://localhost:8080
```

Actualmente no existe autentificacion. Por lo tanto, los endpoints de negocio deben considerarse solo para uso local hasta incorporar Spring Security, identidad y autorizacion.

6.2 Endpoints De Negocio

Metodo	Ruta	Resultado exitoso	Errores controlados
POST	/api/v1/catalog/brands	201 BRAND_CREATED	400 VALIDATION_ERROR / INVALID_BRAND; 409 BRAND_CODE_ALREADY_EXISTS
GET	/api/v1/catalog/brands/{brandId}	200 BRAND_FOUND	400 INVALID_PARAMETER; 404 BRAND_NOT_FOUND

Crear Marca

```
POST /api/v1/catalog/brands
Content-Type: application/json

{
  "code": "coca-cola",
  "name": "Coca Cola"
}
```

Campo	Regla
code	Obligatorio; maximo 50; letras, numeros, guion y guion bajo.
name	Obligatorio; maximo 150 caracteres.

Consultar Marca

```
GET /api/v1/catalog/brands/{brandId}

Ejemplo:
GET /api/v1/catalog/brands/7cb0363e-f978-46dd-80e3-5314a98b4500
```

6.3 Endpoints Operativos

Metodo	Ruta	Uso
GET	/actuator/health	Salud de aplicacion, base, disco, liveness y readiness.
GET	/actuator/info	Informacion operativa; actualmente sin metadatos adicionales.

Estado verificado

En el corte del 2026-07-23, /actuator/health respondio UP y el componente db reporto PostgreSQL UP.

7. Pruebas Y Evidencia

La suite combina pruebas unitarias de aplicacion, pruebas del adaptador REST, persistencia contra PostgreSQL real mediante Testcontainers, integracion HTTP completa y carga del contexto Spring.

Suite	Pruebas	Cobertura principal
CreateBrandServiceTest	2	Creacion activa y rechazo de duplicados.
GetBrandByIdServiceTest	2	Consulta exitosa y not found.
BrandPersistenceAdapterTest	4	Guardar, duplicado, buscar y ausencia.
BrandControllerTest	6	201, 200, 400, 404 y 409 del contrato web.
CreateBrandApiIntegrationTest	3	HTTP a PostgreSQL para crear.
GetBrandByIdApiIntegrationTest	2	HTTP a PostgreSQL para consultar.
ApiResponseDtoTest	4	Fabricas e invariantes de la envoltura.
ApplicationTests	1	Carga completa del contexto Spring.
Total	24	0 fallos, 0 errores, 0 omitidas.

7.1 Comandos De Verificacion

```
.\gradlew.bat test --no-daemon
```

```
# Prueba enfocada
```

```
.\gradlew.bat test `
  --tests "com.odcc.tienda.modules.catalog.adapter.in.rest.BrandControllerTest" `
  --no-daemon
```

Criterio actual de calidad

No se considera terminado un caso de uso hasta comprobar reglas de aplicacion, contrato HTTP y persistencia real cuando corresponda.

8. Operacion Local

8.1 Arranque

```
# Desde la raiz del proyecto
docker compose up -d database

# Desde apps/backend
.\gradlew.bat bootRun --args="--spring.profiles.active=local" --no-daemon

# Verificacion
Invoke-RestMethod http://localhost:8080/actuator/health
```

8.2 Conectividad

Servicio	Host	Puerto	Observacion
Backend	localhost	8080	Perfil local.
PostgreSQL	localhost	5433	Puerto publicado por Compose.
pgAdmin	localhost	5050	Opcional con perfil tools.

JDBC:
jdbc:postgresql://localhost:5433/tienda_abarrotes

Seguridad de configuracion

Las contraseñas se mantienen en .env o variables de entorno. Este documento no contiene secretos. application-local.yml exige POSTGRES_PASSWORD y .gitignore excluye archivos .env.

8.3 Apagado Seguro

```
# Detener el backend con Ctrl+C en su terminal.

# Detener contenedores y conservar los volúmenes
docker compose --profile tools down
```

No se debe usar --volumes al cerrar una sesión normal, porque eliminaría los volúmenes persistentes de PostgreSQL y pgAdmin.

9. Trabajo Pendiente Priorizado

9.1 Prioridad Inmediata - Cerrar Marcas

Codigo	Pendiente	Criterio de terminado
BE-BR-003	Listar marcas con paginacion, filtro y orden.	Limite acotado, contrato documentado y pruebas.
BE-BR-004	Actualizar codigo y nombre.	Validaciones, conflicto por codigo y transaccion.
BE-BR-005	Desactivar y reactivar.	Cambio de estado auditable; sin borrado fisico.
BE-BR-006	Documentar OpenAPI.	Swagger expone request, response y errores.

9.2 Fundacion Transversal Del Backend

Prioridad	Capacidad	Trabajo requerido
P0	Errores globales	JSON malformado, rutas/metodos invalidos y 500 seguro con correlacion.
P0	Seguridad	Spring Security, login, hash de contrasena, tokens, roles y permisos.
P0	Autorizacion	Permisos por caso de uso y pruebas 401/403.
P0	Auditoria de aplicacion	Actor, correlacion y eventos funcionales append-only.
P1	OpenAPI	Contrato navegable y versionado para web y movil.
P1	CORS	Origenes por ambiente antes de integrar Angular.
P1	Pruebas arquitectonicas	Proteger adapter -> application -> domain.
P1	Observabilidad	Logs estructurados, request ID, metricas y trazas.

9.3 Modulos De Negocio

Orden	Modulo	Alcance backend
1	Identidad	Usuarios, roles, permisos, autenticacion y sesiones.
2	Organizacion	Sucursales, almacenes y dispositivos.
3	Catalogo	Categorias, unidades, impuestos, productos, presentaciones, codigos y precios.
4	Inventario	SalDOS, lotes, FEFO, reservas, conteos, movimientos y concurrencia.
5	Compras	Proveedores, ordenes, recepciones, costos y cuentas por pagar.
6	Ventas	Carrito, pedidos, pagos, devoluciones y reservas.
7	Caja y credito	Apertura, cierre, movimientos, limites, cargos y abonos.
8	Facturacion	Datos y estados CFDI 4.0; PAC sigue fuera de alcance.
9	Sincronizacion	Idempotencia, cupos offline, inbox, outbox y conflictos.

10. Hoja De Ruta Recomendada

Fase	Objetivo	Entregables	Salida
F1	Completar administracion de marcas.	Listar, actualizar, activar/desactivar, OpenAPI y pruebas.	CRUD controlado sin borrado fisico.
F2	Asegurar la plataforma API.	Seguridad, errores globales, auditoria, CORS y correlacion.	API apta para clientes internos.
F3	Completar organizacion y catalogo.	Sucursales, almacenes, categorias, productos, presentaciones y precios.	Maestros suficientes para inventario.
F4	Implementar inventario transaccional.	Saldos, lotes, reservas, movimientos y prueba concurrente.	Sin sobreventa en operaciones conectadas.
F5	Implementar compras y ventas.	Recepciones, costos, pedidos, pagos, devoluciones y caja.	Operacion comercial basica.
F6	Integrar clientes y despliegue.	Angular, movil, Docker backend, CI/CD y observabilidad.	Primera entrega integral desplegable.

10.1 Proximo Incremento Concreto

GET /api/v1/catalog/brands
Implementar listado paginado con filtros opcionales por texto y estado, orden estable, limite maximo y respuesta con metadatos de paginacion. Debe incluir pruebas del caso de uso, adaptador JPA, contrato REST e integracion con PostgreSQL.

10.2 Criterios Para No Adelantar Modulos

- No iniciar Angular o movil sobre contratos que aun cambian en cada incremento.
- No implementar ventas antes de probar reservas de inventario bajo concurrencia.
- No exponer entornos compartidos antes de incorporar autenticacion y autorizacion.
- No duplicar reglas del esquema mediante SQL de arranque fuera de Flyway.
- No crear carpetas internas de modulos hasta desarrollar su primer caso de uso.

11. Riesgos Y Acciones De Control

Codigo	Riesgo	Impacto	Mitigacion
RIE-001	La base avanza mas rapido que el backend.	Reglas sin uso o discrepancias de mapeo.	Verticales pequenas con pruebas contra migraciones reales.
RIE-002	API sin autenticacion.	Acceso no autorizado si se publica.	Mantener uso local e implementar seguridad como P0.
RIE-003	Contrato de errores no global.	Respuestas distintas ante fallos no controlados.	Crear advice compartido y pruebas de JSON malformado/500.
RIE-004	Concurrencia de inventario aun no implementada en Java.	Sobreventa al iniciar ventas prematuramente.	Caso de uso atomico y prueba con compradores concurrentes.
RIE-005	Backend fuera de Docker Compose.	Arranque y despliegue manuales.	Dockerfile multi-stage y servicio backend con healthcheck.
RIE-006	Documentacion de backend incompleta.	Ambigüedad de alcance y aceptacion.	Crear requisitos y referencia OpenAPI versionada.
RIE-007	Secretos locales mal gestionados.	Exposicion accidental.	Mantener .env ignorado y usar secretos del entorno en despliegue.
RIE-008	Sin pipeline continuo.	Regresiones no detectadas antes de integrar.	CI con compilacion, pruebas y validacion de migraciones.

11.1 Decisiones Arquitectonicas Vigentes

ADR	Decision	Estado
ADR-0001	PostgreSQL y Flyway como plataforma de datos.	Aceptada
ADR-0002	Inventario offline mediante cupos por dispositivo.	Aceptada
ADR-0003	Auditoria funcional y tecnica en capas.	Aceptada
ADR-0004	Arquitectura hexagonal modular para el backend.	Aceptada

12. Pendientes De Gestion Y Documentacion

Codigo	Pendiente	Responsable	Prioridad
DOC-001	Actualizar PROJECT.md con el avance real del backend.	Por definir	Alta
DOC-002	Crear alcance funcional especifico del backend.	Por definir	Alta
DOC-003	Documentar API mediante OpenAPI y guia de consumo.	Por definir	Alta
DOC-004	Definir responsable del producto y aprobador tecnico.	Por definir	Alta
DOC-005	Aprobar alcance v1.0 de la base de datos.	Negocio / tecnico	Media
DOC-006	Definir criterios de retencion para auditoria y logs.	Seguridad / operaciones	Media
DOC-007	Definir plataforma y estrategia de la aplicacion movil.	Arquitectura	Media
DOC-008	Preparar manual de despliegue cuando exista entorno compartido.	Operaciones	Baja

Proxima decision
Aprobar si el siguiente incremento sera el listado paginado de marcas o si se priorizara primero la seguridad. La recomendacion tecnica es cerrar marcas y, de inmediato, establecer seguridad y errores globales antes de abrir nuevos modulos.

12.1 Referencias Del Proyecto

Documento / ruta	Uso
README.md	Inicio rapido, componentes y decisiones principales.
PROJECT.md	Ficha, tecnologias y proximas acciones.
docs/requirements/database-scope.md	Alcance y reglas de negocio de datos.
docs/architecture/database-model.md	Esquemas, concurrencia y auditoria.
docs/architecture/data-dictionary.md	Definiciones de tablas y campos.
docs/decisions/ADR-0001..0004	Decisiones arquitectonicas aceptadas.
docs/operations/database-docker.md	Operacion local de PostgreSQL.
apps/backend	Codigo y pruebas del backend.

Fin del documento. Estado basado en inspeccion del repositorio, resultados de pruebas, Docker Compose y endpoint Actuator al 2026-07-23.